

Manual de Instalación

1. Introducción

1.1 Propósito y Alcance

- Este manual tiene como objetivo guiar al administrador de sistemas o técnico de IT en la instalación y configuración inicial de la versión 1.0.0 del Sistema de Gestión de Agendamiento (SGA) para la Óptica Unidad Visual.
- El documento cubre la instalación del backend (API Node.js), la base de datos MySQL, el frontend web React, y la configuración inicial del sistema. No cubre el uso funcional del sistema ni el mantenimiento rutinario.

1.2 Audiencia

Este manual está dirigido a:

- Administradores de Sistemas o técnicos de IT responsables del despliegue inicial.
- Desarrolladores del equipo SGA que realicen el primer despliegue en producción.
- Administrador de la Óptica Unidad Visual que gestione el servidor local.

1.3 Convenciones Documentales

- Código / Comando, Comandos que deben ejecutarse en la terminal → npm install
- Negrita, Nombres de archivos, carpetas o campos importantes → env, package.json
- Cursiva, Valores que el usuario debe reemplazar con los suyos → tu_contraseña
- [BOTÓN], Elemento de interfaz de usuario que se debe presionar → [Instalar]
- IMPORTANTE, Advertencia crítica que puede causar fallo si se ignora → Ver sección 3.1

2. Requisitos Previos

2.1 Requisitos de Hardware

Componente	Mínimo Requerido
Servidor de Aplicaciones	4GB RAM, 2 VCPU, 20 GB Almacenamiento
Servidor de Base de Datos	2GB RAM, 10 GB Almacenamiento
Sistema Operativo	Debian 13 Linux o Windows

Servidor Web/Proxy	NGINX
---------------------------	-------

2.2 Requisitos de Software

oftware	Versión Requerida	Propósito	Descarga
Node.js	v18 o superior	Entorno de ejecución del backend (API)	https://nodejs.org
npm	v9 o superior (incluido con Node.js)	Gestor de paquetes de Node.js	Incluido con Node.js
MySQL	8.0 o superior	Motor de base de datos relacional	https://dev.mysql.com/downloads/
Git	Cualquier versión reciente	Clonar el repositorio del proyecto	https://git-scm.com
Navegador Web	Chrome 90+ o Firefox 90+	Acceder al frontend del sistema	chrome.google.com
Sistema Operativo	Windows 10/11 o Ubuntu 20.04+	Compatible con Node.js y MySQL	—

2.3 Recursos Necesarios

Antes de comenzar la instalación, asegúrese de tener disponibles los siguientes elementos:

- Código fuente del proyecto: carpeta apis-sga-completofull (backend) y carpeta sga_optica (frontend).
- Credenciales de acceso al servidor MySQL (usuario root o usuario con permisos de creación de bases de datos).
- Datos de configuración del correo electrónico para notificaciones (cuenta Gmail con contraseña de aplicación habilitada).
- Puerto 3002 disponible para el backend y puerto 5173 disponible para el frontend (en desarrollo).

3. Procedimiento de Instalación

3.1 Instalación en el Servidor (o Estación Principal)

Paso 1: Preparación del entorno:

1. Asegurarse de que tiene acceso a internet y demás herramientas a continuación.
2. Actualizar el sistema desde la terminal:

```
sudo apt update && sudo apt upgrade -y
```
3. Instalar Git, MariaDB y herramientas auxiliares:

```
sudo apt install -y git mariadb-server mariadb-client curl wget
```
4. Instalar Node.js 24.x:

```
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -- Sudo apt install -y nodejs
```
5. Verificamos las versiones y de que todo esté funcionando:

```
git -v  
node -v  
npm -v  
mariadb -v
```
6. Instalar Visual Studio Code:
<https://code.visualstudio.com/docs/?dv=linux64>

Una vez de asegurarse de haber tenido todas estas herramientas se podrá continuar.

Paso 2: Preparación del entorno para el Backend:

1. Crear una carpeta en "files" en la parte de "documents" la cual tendrá como nombre "sga optica".
2. Dentro de la carpeta recién creada crear otras dos carpetas, una con el nombre "front" y la segunda con el nombre "backend".
3. Abrir el Visual Studio Code y en el abrir la carpeta "backend" donde clonaremos el backend de SGA desde la terminal del mismo Visual Studio Code:

```
git clone https://github.com/MarlonCS02/ApiSGAOPTICA.git
```

4. Vas a abrir la terminal de la máquina virtual y una vez allí vas a acceder a "mariadb" como root:

```
sudo mariadb -u root -p
```

5. Creamos la base de datos llamada "sga_optica":

```
create database sga_optica;
```

y Verificamos que se haya guardado en la base de datos:

show databases;

```
MariaDB [(none)]> create database sga_optica;
Query OK, 1 row affected (0,018 sec)

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| app_web |
| information_schema |
| mysql |
| performance_schema |
| sga_optica |
| sys |
| webapp_db |
+-----+
7 rows in set (0,001 sec)
```

6. Crear un usuario dedicado:

```
CREATE USER 'sga_user'@'localhost' IDENTIFIED BY 'sga123';
```

Y otorgar privilegios:

```
GRANT ALL PRIVILEGES ON sga_optica.* TO 'sga_user'@'localhost';
```

Recargar privilegios:

```
FLUSH PRIVILEGES;
```

verificas de que el usuario se haya creado:

```
SELECT User, Host FROM mysql.user;
```

y salimos de MariaDB:

```
EXIT;
```

```
MariaDB [(none)]> SELECT User, host FROM mysql.user;
+-----+-----+
| User | Host |
+-----+-----+
| app_user | localhost |
| mariadb.sys | localhost |
| mysql | localhost |
| node_user | localhost |
| root | localhost |
| sga_user | localhost |
+-----+-----+
6 rows in set (0.039 sec)
```

7. Conectarse con el usuario sga_user:

```
mariadb -u sga_user -p -h localhost;
```

password: sga123

8. Seleccionar base de datos:

```
use sga_optica;
```

9. En Visual Studio Code abrir la terminal en "git bash" donde vamos a limpiar dependencias previas (para evitar conflictos):

(Antes de realizar lo siguiente asegurate de estar en la carpeta correcta la cual es "apis-sga-completofull")

en la terminal poner el comando "cd" para desplazarse entre carpetas:

```
cd ApisSGAOPTICA/
```

```
cd apis-sga-completofull/
```

una vez ya ubicado en la carpeta correcta continuar con la limpieza de dependencias:

```
rm -rf node_modules package-lock.json
```

10. Desde la misma terminal Instalar dependencias:

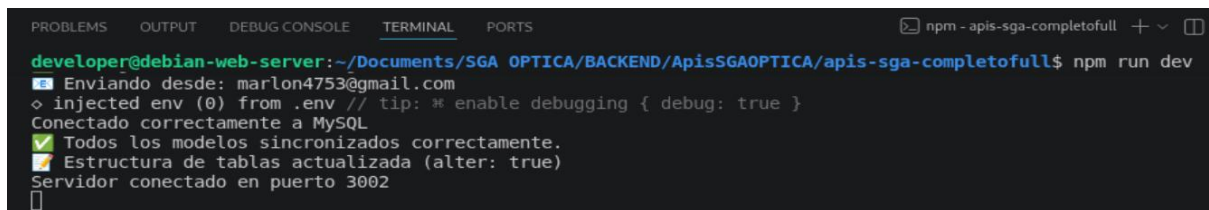
```
npm install
```

11. Buscar la carpeta ".env" y cambiar los siguientes campos:

- En "**DB_NAME**" poner el nombre de la base de datos recién creada "sga_optica".
- En "**DB_USER**" poner el nombre del usuario creado para esta "sga_user".
- En "**DB_PASSWORD**" poner la contraseña también creada recientemente "sga123".

12. Desde la terminal de Visual Studio Code iniciar el servidor:

```
npm run dev
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS npm - apis-sga-completofull + v []
developer@debian-web-server: ~/Documents/SGA OPTICA/BACKEND/ApisSGAOPTICA/apis-sga-completofull$ npm run dev
[+] Enviando desde: marlon4753@gmail.com
> injected env (0) from .env // tip: * enable debugging { debug: true }
Conectado correctamente a MySQL
✅ Todos los modelos sincronizados correctamente.
✅ Estructura de tablas actualizada (alter: true)
Servidor conectado en puerto 3002
[]
```

13. Desde la terminal dentro de MariaDB y de la base de datos sga_optica verificar si se crearon las 15 tablas (punto 7 y 8 para entrar a la base de datos):

SHOW TABLES;

```
MariaDB [sga_optica]> show tables;
+-----+
| Tables_in_sga_optica |
+-----+
| appointment          |
| category              |
| customer              |
| document_type         |
| exam_type             |
| formula               |
| notification          |
| optometrist           |
| payment_type          |
| product               |
| role                  |
| sale                  |
| sale_product          |
| user                  |
| user_entity           |
+-----+
15 rows in set (0,001 sec)
```

Paso 3: Preparación del entorno para el Front:

1. Buscar la carpeta llamada "front" y abrirla en Visual Studio Code la cual fue creada al principio del paso 2 junto con la carpeta del backend.
2. Abrir la terminal de Visual Studio Code y clonar el repositorio del front de SGA:

git clone <https://github.com/sebitaxsz/SGA-WEB.git>

3. Instalamos dependencias:

(Antes de realizar lo siguiente asegurate de estar en la carpeta correcta la cual es "sga_optica")

en la terminal poner el comando "cd" para desplazarse entre carpetas:

```
cd SGA-WEB/
```

```
cd sga_optica/
```

una vez ya ubicado en la carpeta correcta continuar con la actualización de dependencias:

```
rm -rf node_modules package-lock.json
```

```
npm install
```

4. Averiguamos cual es la IP desde la terminal:

```
ip a;
developer@debian-web-server:~$ ip a;
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:02:d8:6f brd ff:ff:ff:ff:ff:ff
    altname enx08002702d86f
    inet 192.168.40.30/24 brd 192.168.40.255 scope global dynamic noprefixroute enp0s3
        valid_lft 83014sec preferred_lft 83014sec
    inet6 fe80::a00:27ff:fe02:d86f/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

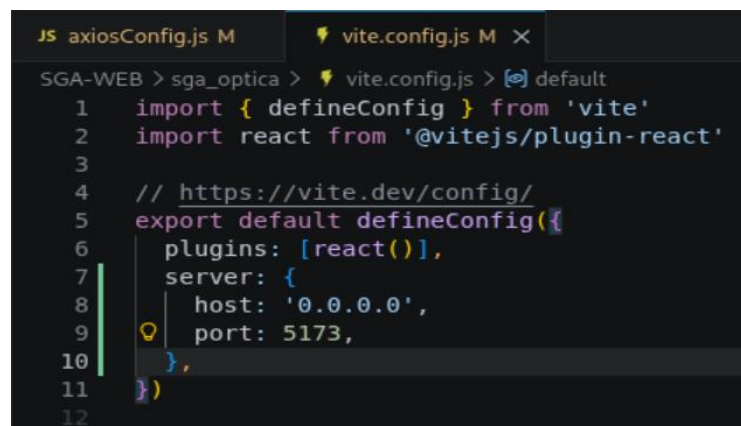
Copiar la que esta al lado de "inet", ej: 192.168.40.30

5. En Visual Studio Code buscar la carpeta llamada "[axiosConfig.js](#)" la cual está dentro de la carpeta de "services" y una vez ahí cambiar la API_BASE_URL:

API_BASE_URL = 'http//192.168.40.30:3002/api/v1';

(Asegúrate de que el "http" este sin la "s" al final o de lo contrario no funcionara).

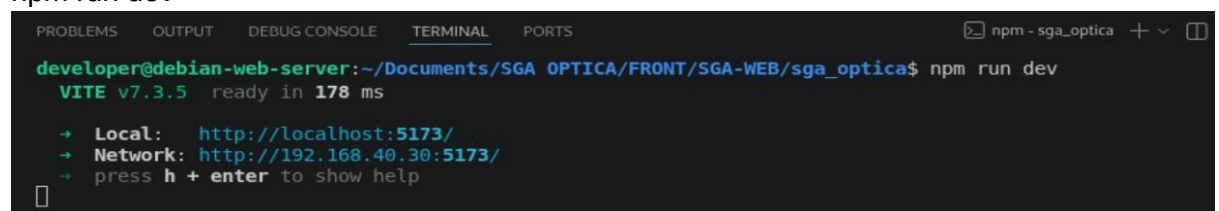
Asegurarse de que la carpeta "vite.config.js" este asi:



```
JS axiosConfig.js M vite.config.js M X
SGA-WEB > sga_optica > vite.config.js > default
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 // https://vite.dev/config/
5 export default defineConfig({
6   plugins: [react()],
7   server: {
8     host: '0.0.0.0',
9     port: 5173,
10  },
11 })
12
```

6. Desde la terminal de Visual Studio Code iniciar el servidor:

npm run dev



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
npm - sga_optica + -
developer@debian-web-server:~/Documents/SGA_OPTICA/FRONT/SGA-WEB/sga_optica$ npm run dev
VITE v7.3.5 ready in 178 ms
→ Local: http://localhost:5173/
→ Network: http://192.168.40.30:5173/
→ press h + enter to show help
```

Paso 4: Cargar los datos iniciales obligatorios:

1. Cargar MariaDB en la base de datos sga_optica:

```
mariadb -u sga_user -p -h localhost;
```

```
password: sga123
```

2. Insertar los roles del sistema:

```
MariaDB [sga_optica]> INSERT INTO role (role_name, role_description) VALUES
-> ('Administrador', 'Control total del sistema'),
-> ('Cliente', 'Viene hacer uso de la pagina'),
-> ('Optometra', 'Revisa a los Clientes');
Query OK, 3 rows affected (0.056 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

3. Insertar los tipos de documento:

```
MariaDB [sga_optica]> INSERT INTO document_type (type_document, document_name) VALUES
-> ('CC', 'Cedula de ciudadania'),
-> ('TI', 'Tarjeta de identidad');
Query OK, 2 rows affected (0.039 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

4. Insertar tipos de pago:

```
MariaDB [sga_optica]> INSERT INTO payment_type (name) VALUES
-> ('Efectivo'),
-> ('Tarjeta de credito'),
-> ('Tarjeta debito');
Query OK, 3 rows affected (0.038 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

5. Asegurarse de que se estén guardando los datos por medio del comando:

```
select * from;
```

```
ej: select * from role;
```

```
MariaDB [sga_optica]> SELECT * FROM role;
+-----+-----+-----+
| role_id | role_name | role_description |
+-----+-----+-----+
| 1 | Administrador | Control total del sistema |
| 2 | Cliente | Viene hacer uso de la pagina |
| 3 | Optometra | Revisa a los Clientes |
+-----+-----+-----+
3 rows in set (0.039 sec)
```

6. Crear un usuario desde la página de SGA ÓPTICA usando el Front.

Vista de la Pagina web:



S.G.A ÓPTICA

Tu visión, nuestra pasión

4. Configuración Post-Instalación

4.1 Configuración de Servicios

Una vez instalados el backend y el frontend, verificar que ambos servicios están activos:

- Backend (API) → **Dominio en 3002** Respuesta JSON con el listado de productos (puede estar vacío inicialmente). Código HTTP 200.
- Frontend (Web) → **Dominio en 5173** Página principal del SGA con el carrusel de imágenes y el catálogo de productos.
- Base de Datos → **SHOW TABLES; (en MySQL)** Deben aparecer las 15 tablas creadas automáticamente por Sequelize: user, customer, appointment, product, etc.

4.2 Configuración de Puertos y Red

Los siguientes puertos deben estar disponibles y abiertos en el firewall del servidor:

- Puerto: 3002, Servicio: Backend — API REST ([Node.js/Express](https://nodejs.org/en/express)), TCP/HTTP Puerto principal de la API. Todos los endpoints del sistema usan este puerto.
- Puerto: 3306, Servicio: Base de datos MySQL, TCP Solo debe ser accesible desde el servidor local (no exponer a internet).
- Puerto: 5173, Servicio: Frontend — Servidor de desarrollo (Vite), TCP/HTTP Solo en modo desarrollo. En producción, usar el puerto 80 o 443 con el build estático.

4.3 Primer Inicio de Sesión y Carga de Datos Iniciales

Al iniciar el sistema por primera vez, la base de datos estará vacía. Seguir estos pasos para dejar el sistema listo para operar:

- 1. Insertar datos de catálogo base:** Ejecutar los scripts SQL de inserción inicial para: roles (ADMIN, EMPLOYEE, OPTOMETRIST, CUSTOMER), tipos de documento (CC, TI, CE, PA, NIT), tipos de pago (Efectivo, Tarjeta débito, Tarjeta crédito, Transferencia) y tipos de examen óptico.
- 2. Registrar el primer usuario administrador:** Usar el endpoint POST `/api/v1/user/register` (o directamente en la BD) para crear el usuario administrador principal con rol ADMIN.
- 3. Acceder al panel de administración:** Abrir el navegador en `http://localhost:5173/login`, ingresar las credenciales del administrador y navegar a `/admin` para acceder al Panel de Administración.
- 4. Registrar optómetras y empleados:** Desde el Panel Admin → AdminOptometras y AdminUsuarios, registrar el personal de la óptica.
- 5. Cargar el catálogo de productos:** Desde Panel Admin → AdminProductos, ingresar los productos del inventario de la óptica con sus precios, categorías y fotos.

5. Verificación y Solución de Problemas (Troubleshooting)

5.1 Prueba de Instalación

Para verificar que la instalación fue exitosa, realizar las siguientes pruebas en orden:

#	Prueba	Cómo verifica	Resultado Esperado
1	Backend arrancó correctamente	Revisar la terminal del backend	Mensaje: 'Server running on port 3002' y listado de tablas sincronizadas por Sequelize
2	BD con tablas creadas	Ejecutar SHOW TABLES en MySQL	15 tablas visibles: user, role, customer, appointment, product, formula, etc.
3	API responde correctamente	Abrir <code>http://localhost:3002/api/v1/products</code> en el navegador	Respuesta JSON: [(lista vacía) o con productos si ya se cargaron. Código 200.
4	Frontend carga en el navegador	Abrir <code>http://localhost:5173</code>	Página principal del SGA visible con navbar y carrusel.
5	Login funciona	Ir a <code>/login</code> e ingresar las credenciales del administrador	Redirección exitosa al panel de administración en <code>/admin</code> .
6	Email de recuperación funciona	Ir a <code>/forgot-password</code> e ingresar el email del administrador	Correo recibido en la bandeja de entrada con el código de recuperación.

5.2 Errores Comunes y Solución

Problema	Mensaje de Error Típico	Solución
No conecta a MySQL	SequelizeConnectionError: connect ECONNREFUSED 127.0.0.1:3306	Verificar que el servicio MySQL está activo. En Windows: Servicios → MySQL80 → Iniciar. En Linux: sudo service mysql start
Credenciales de BD incorrectas	SequelizeAccessDeniedError: Access denied for user 'root'	Revisar los valores de DB_USER y DB_PASSWORD en el archivo .env. Verificar que coinciden con las credenciales de MySQL.
Puerto 3002 ocupado	Error: listen EADDRINUSE: address already in use :::3002	Cambiar el valor de SERVER_PORT en el .env a otro puerto disponible (ej: 3003), o detener el proceso que usa el puerto 3002.
Frontend no conecta al backend	Network Error / Failed to fetch en el navegador	Verificar que el backend está corriendo antes del frontend. Revisar que la base URL en axiosConfig.js apunte a http://localhost:3002/api/v1.
Correo no se envía	Error: Invalid login: 535-5.7.8 Username and Password not accepted	Verificar que EMAIL_USER y EMAIL_PASS en el .env son correctos. Para Gmail, usar una contraseña de aplicación, no la contraseña personal.
Imagen de producto no sube	MulterError: LIMIT_FILE_SIZE o ruta /uploads/ no existe	Verificar que la carpeta src/uploads/products/ existe en el backend. Crear manualmente si es necesario: mkdir -p src/uploads/products
Error al sincronizar modelos	SequelizeDatabaseError: Table already exists	El sistema usa alter:true en syncDatabase(). Si persiste, ejecutar DROP DATABASE api_sga_full; y volver a crear la BD limpia.

5.3 Contacto de Soporte

En caso de fallos no documentados en este manual, contactar al equipo de desarrollo del proyecto SGA:

- **Equipo de Desarrollo:** Juan Casilimas · Wilson Salcedo · William Rojas · Evjany Segura · Marlon Castañeda
- **Versión del sistema:** SGA v1.0.0 — Óptica Unidad Visual